25 FEB 2026

AADK TASK 3

Comprehensive Feature Design & Implementation Strategy

MindMatrix

1. Feature Definition

Feature Name: Bill Split & Tip Tracker

This feature allows the user to:

- Enter total bill amount
- Enter number of people
- Enter custom tip percentage
- Choose round-up option
- Calculate per-person amount
- Save the split to history

Purpose

This feature helps users quickly split bills in restaurants and also track previous splits for reference.

Target Users

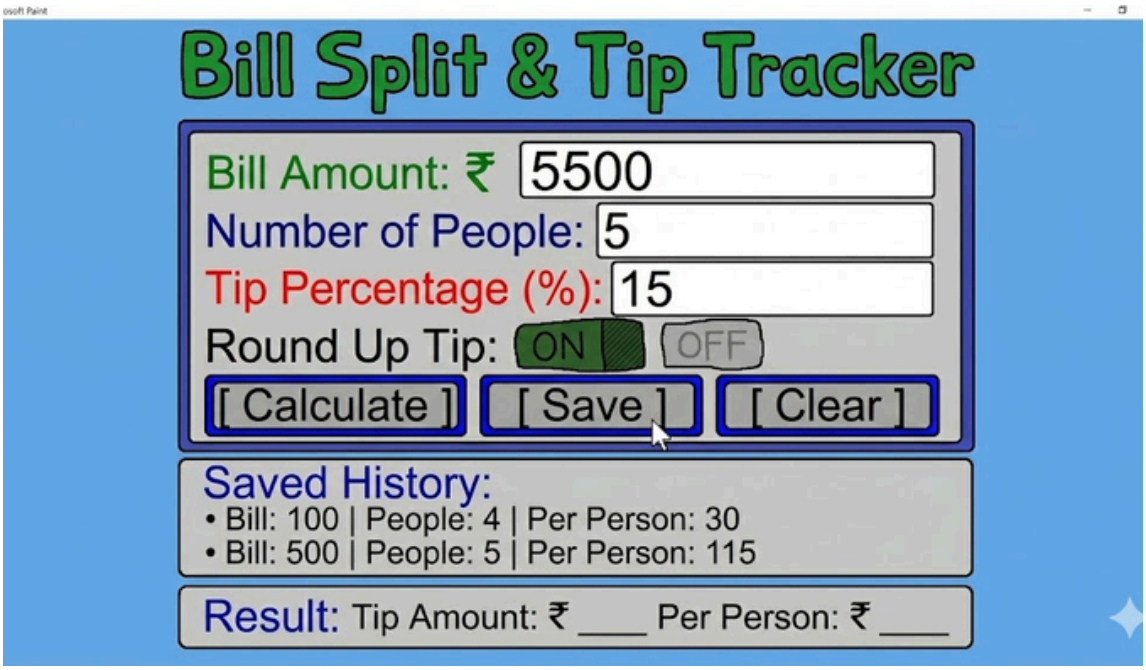
Friends, families, roommates, or anyone who frequently splits expenses.

Key Behaviors

- Input entry (bill, people, tip %)
- Toggle for round-up
- Calculate result
- Save result to history list
- View saved splits

2. Implementation Strategy Document

A. Layout Sketch



Optional: A separate “History Screen” could show full list in LazyColumn.

B. Buttons & Their State Effects

Button	What It Does	State Variables Affected
Calculate	Calculates tip + per person	result, tipAmount
Save	Adds current split to history list	historyList
Clear	Resets all inputs	billAmount, numPeople, tipPercentage, roundUp

C. State Model

State Variable	Type	Mutable?	Initial Value	Notes
billAmount	String	Yes	""	User input
numPeople	String	Yes	""	User input
tipPercentage	String	Yes	"15"	Default tip
roundUp	Boolean	Yes	FALSE	Toggle state
tipAmount	Double	Derived	0	Calculated
perPersonAmount	Double	Derived	0	Calculated
historyList	List	Yes	emptyList()	Stores saved splits

Derived states:

- tipAmount
- perPersonAmount

These depend on billAmount, tipPercentage, numPeople, roundUp.

D. Pseudo-code / Snippet

Calculate Logic

```
val bill = billAmount.toDoubleOrNull() ?: 0.0
val people = numPeople.toIntOrNull() ?: 1
val tipPercent = tipPercentage.toDoubleOrNull() ?: 0.0

var tip = bill * (tipPercent / 100)

if (roundUp) {
    tip = ceil(tip)
}
val total = bill + tip
val perPerson = total / people

tipAmount = tip
perPersonAmount = perPerson
```

Save Action Snippet

```
data class SplitItem(
    val bill: Double,
    val people: Int,
    val tipPercent: Double,
    val perPerson: Double
)

historyList = historyList + SplitItem(
    bill = bill,
    people = people,
    tipPercent = tipPercent,
    perPerson = perPerson
)
```

This adds the current split to the history list.

3. Testing & Edge-Case Plan

Test Case 1 – Valid Input

Input:

- bill = 100
- numPeople = 4
- tip = 20%
- roundUp = false

Expected:

- Tip = 20
- Total = 120
- Per Person = 30

Test Case 2 – Invalid Input

Input:

- bill = 100
- numPeople = 4
- tip = 20%
- roundUp = false

Expected:

- Tip = 20
- Total = 120
- Per Person = 30

Test Case 3 – Large / Edge Case

Input:

- bill = 10000
- numPeople = 0

Expected:

- Show error "Number of people must be at least 1"
- Prevent division by zero
- Button disabled

UI Test Considerations

- Orientation change (screen rotation) → state should not reset
- Keyboard hiding should not remove result
- State should survive recomposition
- Possibly use ViewModel for better state persistence

4. Scaling & Maintenance Considerations

If I want to extend this feature:

- Add support for multiple currencies (₹, $, €)
- Save history permanently using DataStore or Room
- Move calculation logic into a separate function for clean code
- Use ViewModel to manage state properly

To keep code clean:

- Separate UI and business logic
- Keep composables stateless when possible
- Hoist state to top-level

5. Reflection

The most complex part of planning this feature was managing multiple state variables and understanding which ones are derived. I had to think carefully about validation and avoiding division by zero errors.

I now feel more confident handling state in Compose, especially with derived values and lists. I also understood why state hoisting and proper planning are important. Next time, I will clearly separate UI logic and calculation logic from the beginning to avoid messy code.